

Day 52: Add Product to Cart Method

Day 52: Add Product to Cart Method

1. Day Number

Day 52

2. Topic Name

Method with List `append()`

Today you will learn how an object method can receive another object and store it inside a list.

3. Connection

Previously, you created:

- a Product class
- a Cart class
- an empty `items` list inside the cart

Today, you will connect these two classes.

A Product object will be passed to the Cart, and the cart will store that product inside its `items` list.

Conceptually:

```
Product object
  |
  v
add_product()
  |
  v
Cart.items
```

4. Important Topics

Method

A **method** is a function that belongs to a class.

Example structure:

```
class Example:
    def some_method(self):
        ...
```

Object

An object is an instance created from a class.

```
product = Product(...)
cart = Cart()
```

Here:

- product is a Product object
- cart is a Cart object

List append()

append() adds one item to the end of a list.

```
numbers = [10, 20]

numbers.append(30)
```

Now:

```
[10, 20, 30]
```

The important difference today is that instead of adding a number or string, you will add an **object**.

```
cart.items
|
+----- Product object
```

5. Foundational Notes

Suppose your cart starts like this:

```
Cart
└─ items = []
```

Now imagine you create a product:

```
Product
├─ name = "Laptop"
├─ price = 50000
└─ stock = 5
```

You want to place that complete Product object inside the cart.

After adding:

```
Cart
└─ items
    └─ Product("Laptop", 50000, 5)
```

The list is not required to store only simple values.

Python lists can store:

```
[10, 20, 30]
```

or:

```
["Laptop", "Mouse"]
```

or even objects:

```
[Product object, Product object, Product object]
```

This is an important OOP concept.

The method will need a parameter

Your method needs to receive the product being added.

General idea:

```
def method_name(self, something):  
    ...
```

For today's problem:

```
Cart object  
  +  
Product object  
  |  
  v  
add_product(product)
```

Inside the method, the product should be appended to the cart's own list.

Remember:

```
self.items
```

means:

| the items list belonging to this particular Cart object.

6. Easy Example

Consider a different example: a Bookshelf.

```
class Bookshelf:  
    def __init__(self):  
        self.books = []  
  
    def add_book(self, book):  
        self.books.append(book)
```

Usage:

```
shelf = Bookshelf()

book_name = "Python Basics"

shelf.add_book(book_name)

print(shelf.books)
```

Output:

```
['Python Basics']
```

The important idea is:

```
value
 |
 v
method(value)
 |
 v
list.append(value)
```

Your Cart will follow the same idea, except the value being added will be a **Product object**, not just a string.

7. Problem Statement

Create an `add_product()` method inside the `Cart` class.

The method should:

1. Accept one `Product` object.
2. Add that `Product` object to the cart's `items` list using `append()`.
3. Create one product.
4. Create one cart.
5. Add the product to the cart.
6. Print the cart's items.

Do not create complicated quantity, stock, or price calculations yet.

Focus only on:

```
Product object → Cart → items list
```

8. Concepts Used

You will use:

- class
- object
- constructor `__init__()`

- self
- instance attributes
- list attribute
- method
- method parameter
- append()
- object passed as an argument

The new important concept is:

| **One object can be passed to another object's method.**

For example:

```
cart.add_product(product)
```

Here:

- cart is one object
- product is another object
- the product object is passed into a method belonging to cart

9. Thought Process

Think about the problem in small steps.

Step 1: What objects do I need?

You need:

```
Product
Cart
```

Step 2: What should Product contain?

From Day 50, a Product can contain information such as:

```
name
price
stock
```

Step 3: What should Cart contain?

From Day 51:

```
items = []
```

Initially:

```
Cart
└─ items = []
```

Step 4: What behavior does the Cart need?

The cart needs an action:

```
add_product()
```

Step 5: What information does the method need?

It needs to know:

```
Which product should I add?
```

So the Product object must be passed as an argument.

Conceptually:

```
cart.add_product(product)
```

Step 6: What should happen inside the method?

Take the received product:

```
product
```

and append it to:

```
self.items
```

Step 7: What happens after adding?

Before:

```
items = []
```

After:

```
items = [Product object]
```

10. Pseudocode

```
CREATE Product class

    CREATE constructor
        STORE name
        STORE price
        STORE stock

CREATE Cart class

    CREATE constructor
        CREATE empty items list
```

```
CREATE add_product method that receives a product
  ADD the received product to items list
```

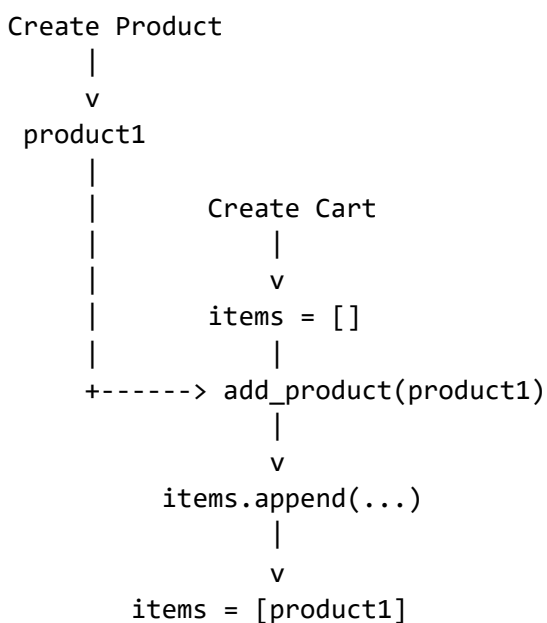
```
CREATE one Product object
```

```
CREATE one Cart object
```

```
CALL cart's add_product method
PASS the Product object
```

```
PRINT cart items
```

Visual flow:



11. Suggested Solving Approach

Use an **OOP-based approach**.

Keep responsibilities separate.

Product class

Responsible for product information.

```
Product
├── name
├── price
└── stock
```

Cart class

Responsible for cart information and cart behavior.

```
Cart
├─ items
├─ add_product()
```

This is better than putting all the logic into unrelated variables because each class has a clear responsibility.

12. Easy Edge Cases

Case 1: Empty cart before adding

Initially:

```
items = []
```

This is completely valid.

After adding one product:

```
items = [Product object]
```

Case 2: Adding one product

Suppose:

```
Product:
name = "Mouse"
price = 500
stock = 10
```

Before:

```
Cart
├─ []
```

After calling:

```
add_product(mouse)
```

Conceptually:

```
Cart
├─ [
    Mouse Product object
  ]
```

For now, you do **not** need to handle:

- duplicate products
- product quantities
- stock reduction

- invalid products
- removing products

Those can be added later.

13. Expected Input

For today's simple exercise, you do not need `input()` from the keyboard.

You can create values directly in your program.

Example data:

```
Product name: Laptop
Price: 50000
Stock: 5
```

Then create:

```
one Product object
one Cart object
```

and pass the Product object to:

```
add_product()
```

14. Expected Output

Before adding:

```
[]
```

After adding, the cart should contain **one Product object**.

If you print the list directly, Python may show something similar to:

```
[<__main__.Product object at ...>]
```

That is normal.

It means:

| The list contains a Product object.

You haven't yet taught Python how to display Product objects in a prettier format.

The important result for Day 52 is:

```
Before:
[]

After:
[Product object]
```

15. Hint Only

Your Cart already has something similar to:

```
self.items = []
```

Now think about these three pieces:

```
def add_product(self, _____):
```

Inside the method, you need:

```
self.items._____(product)
```

And when using the method:

```
cart._____(product)
```

The list method you learned today is:

```
append()
```

So your core thought should be:

```
Receive Product object
    ↓
Access this cart's items list
    ↓
Append Product object
```

Try solving it without adding any extra logic. **Day 52 is mainly about understanding that a list attribute can contain objects and that a method can receive another object as an argument.**